

# Benchmarking Open-Source Property-Graph Databases with a Portable Mixed-Workload Protocol and a Live Scale-Out Event

Mustafa Can Çalışkan  
Computer Engineering Department  
Istanbul Technical University  
Istanbul, Türkiye  
caliskanmu20@itu.edu.tr

**Abstract**—Property-graph database systems differ greatly from one another in terms of storage engines, query languages, transaction mechanisms and elasticity methods. The public literature benchmarking them focuses on specific workloads; no existing portable multi-system benchmark can test them all at the same time when serving both transactional and analytical graph queries at the same time, adding nodes concurrently to an operation or handling both transactional and analytical query concurrency, as would be needed for a real deployment. We propose such a portable multi-system benchmark to test the performance and scale-out behavior of open-source property-graph systems. Reusing subsets of queries from three public LDBC datasets — SNB Interactive v2, SNB Business Intelligence (BI) and FinBench — we define five scenarios targeting isolation of transactional queries, isolation of analytical queries, a mixed regime, a financial transactional regime and a controlled scale-out event that triggers during a mixed transactional-analytical load. The protocol has been implemented as an open-source harness which targets five popular property-graph systems — NebulaGraph, ArangoDB, Dgraph, Memgraph and OrientDB. The harness features a single deterministic closed-loop driver, a suite of per-system bulk loaders, a set of per-system query translations and Docker-based per-node deployment, controlled over SSH. We report how a chosen cluster size — a five-node Google Compute Engine cluster — and the smallest scale factor accepted by each datagen behave under throughput and tail-latency tests. We analyze how well the systems mix transactional and analytical queries, quantify the interference introduced by co-scheduled analytical query processing and measure the time taken for the system to stabilize its performance after adding a new node while mixed queries are executing and the resulting dip in throughput.

**Index Terms**—graph databases, benchmarking, LDBC, scale-out, HTAP, mixed workloads

## I. INTRODUCTION

The common availability of distributed property-graph databases reflects current production practices that position graphs alongside relational and document databases. However, the differences are stark. Storage engines vary, moving from native adjacency-page layouts to LSM-trees on key-value stores. The query interfaces vary as well: nGQL, AQL, DQL, Cypher over Bolt, OrientSQL over REST. The transactional guarantees are wide-ranging, from full-cluster eventual consistency down to per-shard ACID guarantees. Methods for adding

a new node to a cluster differ, from a simple cluster-update call to a manual rebalance. These differences can be important and impact production decisions, but it is difficult to compare the systems in practice because benchmarks divide workload types.

The Linked Data Benchmark Council (LDBC) is a recognized source for workload definitions in graph databases. Its Social Network Benchmark (SNB) Interactive workload [3] describes transactional reads and updates on a temporal social graph. The SNB Business Intelligence (BI) workload [1] describes analytical queries on the same graph schema. FinBench [2] introduces a transactional graph benchmark over financial data. Each benchmark is useful for its target workload, but each only covers one type. Real-world applications involve multiple types. More recent system papers such as Galaxybase [4], G-Tran [5], ByteGraph [6] and BACH [7] motivate the need for hybrid transactional and analytical processing (HTAP) on graph data. However, each paper introduces a custom system and provides an evaluation that cannot be transferred to another system. Each of these papers also omits reporting any event where a node is added while a mixed workload is running and measuring interference during the rebalance. Elastic scale-out is a distinguishing feature of distributed graph systems, yet it is not benchmarked cross-system.

Therefore, there is an information gap for both practitioners and researchers. An engineer choosing an open-source property-graph system for an application requiring many short transactional reads, several updates and longer analytical traversals, with data growing over time, cannot find reproducible comparisons across systems in existing research. A researcher cannot refer to a protocol for benchmarking the combined effects of LDBC workload components, mapping them to different system query languages, specifying scale-out trigger and procedure, or listing comparable performance metrics for mixed-workload interference and elasticity.

This paper seeks to address that gap. Our contribution is three deliverables.

- 1) **A portable evaluation protocol.** For our workloads, we use the seven short reads (*IS1–IS7*), four complex

reads (*IC1*, *IC3*, *IC5*, *IC6*) and three updates (*IU1*, *IU2*, *IU6*) from LDBC SNB Interactive v2 [3]. The analytical workload uses four power-class reads (*Q1*, *Q2*, *Q6*, *Q11*) from LDBC SNB Business Intelligence [1]. For the financial workload, we take six simple reads (*SR1*–*SR6*) and three writes (*W1*–*W3*) from LDBC FinBench [2]. The workloads define five scenarios: a purely transactional workload, another purely analytical one, a mixed regime that mixes short reads, complex reads and updates with a 70 : 20 : 10 ratio, a financial transactional workload and finally a mixed regime with one scale-out operation in the middle of the workload execution. For each scenario, each repetition has distinct, deterministically generated client seeds and we stick with a deterministic client seed setting, a fixed number of repetitions, a warm-up period, a measurement period, a set of common summary metrics and also two distinct metrics for the scale-out scenario, the stabilization time and the throughput dip.

- 2) **An open-source reference implementation.** We use five open-source property-graph systems — specifically NebulaGraph [8], ArangoDB [9], Dgraph [10], Memgraph [11] and OrientDB [12] — to test the implemented protocol. Our strategy is to use one dedicated bulk-loader for each relevant LDBC dataset per database, construct per-system query templates validated against the LDBC benchmark rules for each graph system, include one single, identically structured asynchronous driver that runs in closed loop, use Docker Compose for system deployment, which includes startup, scale-out and removal by using SSH access from a single orchestrator node and output per-run results comprising Parquet files for all client requests, HDR latency histograms per query class, per-second throughput totals, /proc resource consumption on a per-node basis and an `events.jsonl` marking the boundary of phases within the benchmark. The code is public.<sup>1</sup>
- 3) **A first cross-system measurement study.** We complete the campaign on a five-node Google Compute Engine cluster using the lowest scale factor accepted by every datagen. We report, per scenario, throughput broken down by query class and tail latency. For S3, we note the impact of simultaneous mixed workloads through the interference index. For S5, we quantify the time needed to become fully stable after the new node joins and the resulting throughput dip. All five systems run S1; the three Tier-1 systems run S2–S5.

The campaign covers 51 runs and 444,297 requests. ArangoDB is the fastest system in the S1 transactional baseline, approximately 7.6 times faster than NebulaGraph, and this ranking holds across S2–S5 as well; Dgraph falls between the two throughout. For mixed-workload interference (S3), ArangoDB shows a small positive effect from cache sharing, while Dgraph is the only system with measurable slowdown

under the co-scheduled analytical load. In the scale-out scenario (S5), NebulaGraph registers the new node the fastest and with the smallest throughput dip; ArangoDB and Dgraph drop more during the rebalance window and neither system recovers fully within the 60s measurement window.

The remaining sections organize the paper into the following parts. Section II splits related work into four categories, including LDBC benchmarks, HTAP graph systems, the graph engines we evaluate and earlier multi-system graph benchmarking efforts. Section III defines our benchmarking protocol. Section IV describes how we carry out the benchmark harness. Section V lists the system configuration in the Google Compute Engine cluster and the installed versions. Section VI reports the quantitative benchmarking results. Section VII investigates cross-system performance patterns from the results. Section VIII discusses threats to validity. We conclude in Section IX. Wide figures are stored in Appendix A for easier reading.

## II. RELATED WORK

The prior work most relevant to this paper falls into four main categories: first, the LDBC benchmarks used in this work; second, modern single-system research on hybrid transactional-analytical graph processing (HTAP); third, the graph engines that we evaluate; and fourth, previously published multi-system graph benchmark studies.

### A. LDBC Benchmark Family

LDBC SNB Interactive v2 [3] is the current release of the standard for OLTP on the social-network graph schema. Compared to SNB Interactive v1, it has new, stronger features: in addition to richer delete functionality, it guarantees more care on the parameters that are shipped to the workload driver, larger scale factors, a better workload-scheduling process and a cleaner auditing system that includes a required 95% on-time constraint and an audit that includes full disclosure of the results. It defines the three query types we study here: short reads, complex reads and updates. The closed-loop design and the seed discipline are taken from its workload recommendations: on a particular (system, scenario, repetition) trial, every parameter draw is seeded with a particular random seed. But we do not consider this to be SNB Interactive v2 compliance in general, since we only use a portable subset and clearly say so.

LDBC SNB BI [1] is the analytical sibling of SNB Interactive. It also uses the SNB social-network schema. SNB BI is structured around so-called “choke points”—operations that are tricky for graph databases, which it categorizes as pattern matching, graph-aware joins, path finding, historical updates, skewed data and other advanced analytics. It defines measures for power and throughput under sustained workload. The BI paper reports portability on two system prototypes: an implementation using a relational database engine and a second based on a graph DBMS. SNB BI contributes our read-query subset to this protocol. We again consider only a portable subset of its read queries rather than full BI

<sup>1</sup><https://github.com/canetizen/pdgraph-bench>

compliance, as the protocol must run on five systems that use widely different query languages.

LDBC FinBench [2] follows the LDBC tradition outside the domain of social networks and captures the critical challenges of financial graphs: a high degree of edge multiplicity, time-series skewedness at endpoints, hub vertices, edge filtering with time-window restrictions, query paths restricted to follow paths that are recursive, read/write patterns where the query’s read behavior depends on its most recent write and others. FinBench defines our S4 scenario, a non-trivial read-write mixture. Beyond S4 itself, FinBench serves a more general purpose: a system performing well on the SNB social network but poorly on FinBench exhibits a domain-specific deficiency rather than a general weakness in how it handles any given class of graph query operations.

### *B. HTAP and Mixed-Workload Graph Systems*

Galaxybase [4] most closely resembles the HTAP mixed-workload behavior we measure. It relies on a novel Log-Structured Adjacency List that efficiently accommodates transactional inserts and deletes and uses an edge-page structure to organize data. Galaxybase combines elements of 2PC, Raft, 2PL and MVCC to enable both OLTP and OLAP operations on the same distributed database. Galaxybase sets a new benchmark by being one of the first publicly presented systems to explicitly show the value proposition of HTAP-style graph processing, arguing that OLTP and OLAP work should be run together and that the side effects that transactions have on OLAP performance should be accounted for in benchmarks. We adopt its perspective of considering both transaction types together, but we generalize its evaluation methodology beyond a single-system paper into an open-source standard that other systems can evaluate portably against a wide variety of common graph data and workload classes.

G-Tran [5] is a fully distributed high-throughput graph database built on a decentralised transactional design. It achieves high transaction rates in the distributed setting. Nevertheless, its evaluation never puts transactions next to any parallel OLAP query. ByteGraph [6] covers a large-scale production OLTP and OLAP hybrid graph system at ByteDance and sheds light on common pitfalls from its deployment, although again only as a single-system paper. BACH [7] combines adjacency-list and CSR representations with an LSM-tree backing to run an HTAP mixed workload and reports its performance breakdown. All three papers show the importance of mixed-workload graph analytics, yet no public benchmark suite enables fair comparison between systems under this regime. We intend to address this oversight.

### *C. Open-Source Property-Graph Engines*

This paper compares five open-source property-graph engines. Three of these engines are mainstream and have established system papers or high-quality documentation upon which we draw. NebulaGraph [8] is a native graph database that uses its own nGQL query language. It is organized around a distributed architecture of distinct storages, metad

and graphd processes that interact via a host-local Raft protocol. ArangoDB [9] is a multi-model NoSQL engine offering document, graph and key/value storage models unified by the AQL query language. In cluster deployments, it operates with distinct Coordinator, Agent and DBServer roles. Dgraph [10] is a native distributed graph database using a bespoke DQL that shards predicates across multiple alpha processes and communicates with a separate zero process for cluster management.

The remaining two, classified as Tier-2 engines, offer simpler, single-server configurations to make them easy to set up. Memgraph [11] is a fast, in-memory graph engine built with C++. It provides access through the Bolt protocol using Cypher queries. For test-harness interaction, we use the official `neo4j` Bolt driver to connect to Memgraph. OrientDB [12] is another multi-model NoSQL engine, written in Java. It supports native graph capabilities, along with document, full-text search and geo-spatial querying. OrientDB is accessed through an HTTP endpoint and a JSON REST body, where OrientSQL serves as the query language. The harness communicates with OrientDB over HTTP. The five-SUT (System Under Test) comparison set is thus rounded out by Memgraph and OrientDB at the opposing ends of the storage spectrum (purely in-memory Cypher versus a hybrid document-graph store), representing two single-server-per-process engines that balance out the three Tier-1 cluster systems.

### *D. Earlier Multi-System Graph Benchmarks*

A handful of previous academic papers have evaluated and compared graph databases across multiple systems. These studies have, for the most part, been restricted to either one type of workload, a particular graph schema or did not attempt to control the dynamics of an elasticity event within their measurement window. Where applicable, we adopt their techniques in selecting query classes and in structuring the reporting. However, the combination of three features appears unprecedented: a general-purpose, mixed workload assembled from well-established LDBC components, a deterministic, closed-loop query driver used to interact with five property-graph engines that are fundamentally different in their query languages and internal storage layouts, and a measured “scale-out” event occurring during the execution of that same mixed workload. This confluence of factors seems to be lacking in the existing published literature on graph database benchmarks.

## III. BENCHMARK PROTOCOL

This benchmark protocol is composed of five main sections. First, the set of systems under test (SUTs). Second, the datasets used in the protocol. Third, the workload subset that, in concert with per-system translation, defines the benchmark’s execution. Fourth, the set of scenarios that combine those classes. Lastly, the set of performance metrics derived from each scenario run.

### *A. Systems Under Test*

Five open-source property-graph database systems are targeted in this protocol. These systems are arranged in two

TABLE I  
SYSTEMS UNDER TEST AND TIER COVERAGE.

| System      | Query language   | Tier |
|-------------|------------------|------|
| NebulaGraph | nGQL             | T1   |
| ArangoDB    | AQL              | T1   |
| Dgraph      | DQL              | T1   |
| Memgraph    | Cypher (Bolt)    | T2   |
| OrientDB    | OrientSQL (REST) | T2   |

tiers. Tier-1 systems are responsible for running the entire set of benchmark scenarios, whereas Tier-2 systems will only undergo testing in the cross-tier transactional baseline (S1). Table I provides an overview of the chosen systems and their tier coverage. This tiering reflects the range of scenario coverage and not the system’s overall quality.

#### B. Datasets

- 1) We use three publicly available LDBC-family datasets: LDBC SNB Interactive v2 [3] (transactional, social network), LDBC SNB Business Intelligence [1] (analytical, same schema) and LDBC FinBench [2] (transactional, financial-transaction graph).
- 2) Each dataset is created once per scale factor by the LDBC datagen container.
- 3) We use a multi-stage Dockerfile for FinBench to build the FinBench Datagen jar from upstream source, as LDBC does not provide a pre-built image.
- 4) The bulk load runs once for each combination of System Under Test (SUT) and dataset and we exclude load time from performance results.

#### C. Workload Subset and Query Translation

To ensure the workload translates between query languages, we selected specific parts of each dataset:

- 1) LDBC SNB Interactive v2: 7 short reads (IS1–IS7), 4 complex reads (IC1, IC3, IC5, IC6) and 3 updates (IU1, IU2, IU6).
- 2) LDBC SNB BI: 4 power-class reads (Q1, Q2, Q6, Q11).
- 3) LDBC FinBench: 6 simple reads (SR1–SR6) and 3 writes (W1–W3).

We wrote each query template manually in the respective system language and checked it against the LDBC specification, but we do not claim to achieve semantic equivalence with a reference LDBC implementation. The templates are provided as plain text in the repository for review.

We introduced two deviations from the original queries:

- 1) NebulaGraph’s Q1 requires an index on post length. We added this index to the schema rather than altering the query.
- 2) Dgraph does not permit double expansion of the same edge predicate within a sub-query. Therefore, we simplified the IC1 query into a one-hop expansion and a name filter.

Both of these modifications maintain the original intent of the LDBC query.

TABLE II  
FIVE-SCENARIO PROTOCOL.

| ID | Workload composition                       | Dataset     | SUTs  |
|----|--------------------------------------------|-------------|-------|
| S1 | IS1–IS7, IC1, IC3, IC5, IC6, IU1, IU2, IU6 | SNB Iv2     | all 5 |
| S2 | IC1, IC3, IC5, IC6, Q1, Q2, Q6, Q11        | SNB Iv2, BI | T1    |
| S3 | 70 % IS, 20 % IC, 10 % IU                  | SNB Iv2     | T1    |
| S4 | SR1–SR6, W1–W3                             | FinBench    | T1    |
| S5 | S3 mix + scale-out at $t=10$ s             | SNB Iv2     | T1    |

#### D. Scenarios

- 1) Five scenarios were defined to capture different positions along the transactional-vs-analytical and static-vs-elastic axes.
- 2) Table II outlines the workload, dataset and SUT coverage for each scenario.
- 3) The S5 scale-out trigger is activated 10 seconds into the measurement window.

#### E. Workload Protocol

Each test run comprises a seven-step process. First, we boot the SUT cluster across all specified SUT-role nodes using Fabric and SSH. Next, the dataset for the scenario is bulk-loaded using the SUT’s built-in client API. Third, we run a minimal set of post-load validators on known-answer records. Fourth, we run a  $W=10$  s warm-up phase, during which we discard the collected samples. Fifth, we execute a  $M=30$  s measurement phase and record its sample responses, although for S5 we run this for 60 s so that we can view the post-event activity. Sixth, only when running the S5 test, the scale-out event is fired at a configured time offset. Finally, the SUT cluster is shut down.

The workload is run as a closed loop. Sixteen asynchronous workers are spawned per test run (for S2 we spawn only eight because it is purely an analytical benchmark) and each worker sends the next query immediately upon completion of the prior one, meaning there is no user think time. The per-query timeout is 60 s. Queries per worker are selected using a weighted random sampling of query classes per the scenario mix. Per-request parameters are chosen randomly from pools of data for each class, which we generate dynamically in our benchmark process by parsing vertex CSVs (we do not use the provided parameter-curation files for LDBC, which is another variation from LDBC’s gold-standard spec, as noted in Section VIII). Seeding is done independently for every test run (across all system, scenario and repetition counts), ensuring that every parameter draw from a random distribution is perfectly deterministic.

#### F. Metrics

We report seven metrics per run.

- **Throughput.** Operations per second, both in total and per query class, sampled at 1 s intervals.

- **Latency.** Per-request latency, collected into HDR histograms; we report  $p50$ ,  $p95$ , and  $p99$  per class.
- **Mixed-workload interference index.** A scalar that compares the mixed-run throughput to the weighted sum of the isolated per-class throughputs:

$$I = 1 - \frac{T_{\text{mixed}}}{\sum_c w_c T_c^{\text{iso}}} \quad (1)$$

$T_{\text{mixed}}$  is the aggregate ops/s of the mixed scenario,  $w_c$  is the request-share of class  $c$  in the mix, and  $T_c^{\text{iso}}$  is the ops/s for class  $c$  measured in isolation.  $I > 0$  means real interference;  $I < 0$  means a mix-favourable effect;  $I \approx 0$  means the mix neither helps nor hurts.

- **Scale-out provisioning time.** Wall-clock seconds from the `scale_out_trigger` event to the `scale_out_complete` event in the run’s phase log.
- **Scale-out stabilization time.** Seconds since the trigger at which the rolling-average throughput re-enters the band  $[0.95, 1.05] \cdot T_{\text{pre-event}}$  and stays inside it for at least 5 consecutive seconds.
- **Scale-out throughput dip.** Maximum relative throughput drop during the rebalance window, expressed as a percentage of  $T_{\text{pre-event}}$ .
- **Failed-query rate.** Fraction of requests that return an error or exceed the 60s timeout.

We collect per-node resource samples (CPU, memory, disk, network) from the SUT nodes via SSH every 5s by reading `/proc/stat`, `/proc/meminfo`, `/proc/diskstats` and `/proc/net/dev`. These samples are saved in the per-run artifact bundle and summarized in Section VII (Table V).

#### IV. IMPLEMENTATION

This section details the design choices contributing to the benchmark’s portability, reproducibility and ease of rerunning on any five-node Linux cluster. The benchmark harness is written in Python 3.11 and is open source.

##### A. Driver Abstraction and Closed-Loop Harness

We present each SUT as a small driver interface featuring only four methods: connect, disconnect, execute a request and register a new node (exclusively for the S5 scenario). The benchmark runner is unaware of the identity of the SUT it interacts with. Query templates, specific to each SUT, reside alongside the workload definitions and are dynamically loaded when a request is initiated. This enables the use of the same benchmark runner with five different query languages without requiring any modification to the code.

The runner employs a closed-loop design. For each benchmark phase, it asynchronously launches a pool of workers. Each worker selects the next query class from a weighted scheduler, retrieves the corresponding parameters from the workload definition, transmits the request via the driver and records the outcome with the metrics collector. The worker pool continues until the phase deadline is reached. Seeds for per-(system, scenario, repetition) are fixed, ensuring that

individual runs can be reproduced byte for byte. Separate seed namespaces are used for the warm-up and measurement phases to prevent a known collision issue where workloads dynamically add vertices, leading to shared IDs.

##### B. Per-System Deployment

The deployment layer disseminates a per-system Docker plan to each SUT-role node over SSH. This plan outlines the containers, including their images, environments, ports, volumes and health checks. The S5 scenario expands this plan by incorporating an additional container on the reserve node. The benchmark runner’s interaction with this layer is limited to two operations: initiating the cluster and scaling it out. All other deployment complexities are abstracted away by these operations. Consequently, incorporating a new SUT into the benchmark involves a confined, isolated task, leaving the runner and metrics layer untouched.

##### C. Reproducibility Artifacts

Each benchmark run produces a self-contained bundle that encapsulates all essential information for reviewers to replicate or further analyze the run. The bundle contains:

- A per-request log (one line per request, including timestamps, latency, worker ID, query class, status and optionally a result hash).
- Per-class HDR latency histograms.
- One-second throughput buckets per class.
- Per-node resource samples (CPU, memory, disk, network).
- A phase-boundary event log that marks warm-up, measurement and scale-out events.

The complete source code, scenario YAML files, per-system query translations and Docker bundles are publicly available.<sup>2</sup>

#### V. EXPERIMENTAL SETUP

We run our experiments on a five-node Google Compute Engine (GCP) cluster [13]. The entire test campaign is conducted on Google Cloud and no institutional hardware is used.

All five nodes have identical hardware specifications: 2 vCPUs, 8 GiB RAM and a 50 GB balanced persistent disk. The operating system is Ubuntu 24.04 LTS and Docker is installed on each node. The five nodes share a single virtual private network and resolve each other using short hostnames via Google Cloud’s internal DNS service. The cluster resides in a European zone (Belgium), ensuring network latency between nodes of less than one millisecond.

The cluster roles are assigned as follows: node 1 acts as the orchestrator and benchmark driver; nodes 2, 3 and 4 are the SUT nodes; node 5 serves as the reserve node and is used exclusively during the S5 scale-out event.

The absolute throughput figures presented in this paper represent relative performance comparisons among SUTs and should not be interpreted as indicative of their production capacity.

<sup>2</sup><https://github.com/canetizen/pdgraph-bench>

We employ the smallest scale factor supported by each LDBC datagen. For SNB Interactive v2 and SNB BI, this is  $SF=0.003$ ; the SNB datagen produces an invalid output below this scale. For FinBench, the smallest supported scale factor is  $SF=0.01$ ; its datagen rejects any value smaller than this. At these scales, the SNB Iv2 graph contains approximately 1.5k persons, 150k posts and 150k comments. The FinBench graph has roughly 12k vertices and 120k edges. Given this size, the entire dataset for any single (system, dataset) pair fits in the memory of one SUT node, making disk I/O a secondary performance bottleneck.

Each (system, scenario) cell is run three times. S1–S4 feature a 30s measurement window, while S5 uses a 60s window to capture post-event recovery. The entire set of runs is completed in under two hours of wall-clock time.

## VI. RESULTS

We run the protocol on the five-node Google Compute Engine cluster that we describe in Section V. We use the smallest scale factor that each generator accepts:  $SF=0.003$  for SNB Interactive v2 and SNB BI, and  $SF=0.01$  for LDBC FinBench. Each (system, scenario) cell runs three times. This gives 51 runs in total: three Tier-1 systems run all five scenarios with three repetitions, and two Tier-2 systems run S1 with three repetitions. The harness records 444,297 individual requests across the campaign. Every run returns at least 99.98 % successful requests (Dgraph accounts for the tiny error fraction; all other SUTs reach 100 %), no worker dies, and no run is marked as failed by the campaign script.

Table VI (Appendix A) reports the headline throughput and tail-latency numbers per (system, scenario) pair. Throughput is the operations per second over a 30 s measurement window with 16 async workers (8 for the analytical S2 scenario, as the workload specification requires). The latency percentiles come from the per-request log, only over the successful subset.

### A. Cross-tier transactional baseline (S1)

S1 is the only scenario that all five SUTs run, so it is the best scenario for a cross-tier comparison. Fig. 1 shows the throughput. Fig. 2 shows the latency percentiles on a log axis.

The per-class breakdown in Table VII and in Fig. 3 (both in Appendix A) shows that the cross-system gap is almost the same across all 14 query classes of the SNB Interactive v2 subset. ArangoDB is the fastest on every class. NebulaGraph is the slowest on every class. No SUT shows a class-specific weakness or strength at this scale factor.

### B. Tier-1 cross-scenario behaviour (S1–S5)

The three Tier-1 systems run all five scenarios. Fig. 4 (Appendix A) plots their throughput across the scenarios. Fig. 5 (Appendix A) plots the p99 latency on a log axis.

A few things stand out from these plots. First, the throughput is quite stable across scenarios for each Tier-1 system. The coefficient of variation across scenarios is below 15 % for every Tier-1 SUT. This means that at  $SF=0.003$  the bottleneck is the client-server round trip and not the per-class query

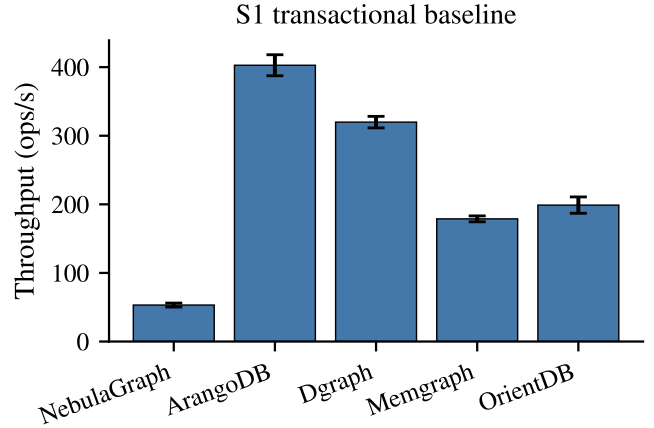


Fig. 1. S1 transactional baseline throughput. Error bars are the standard deviation across three repetitions. ArangoDB is the fastest, around  $7.6\times$  over NebulaGraph. OrientDB and Memgraph form a middle band.

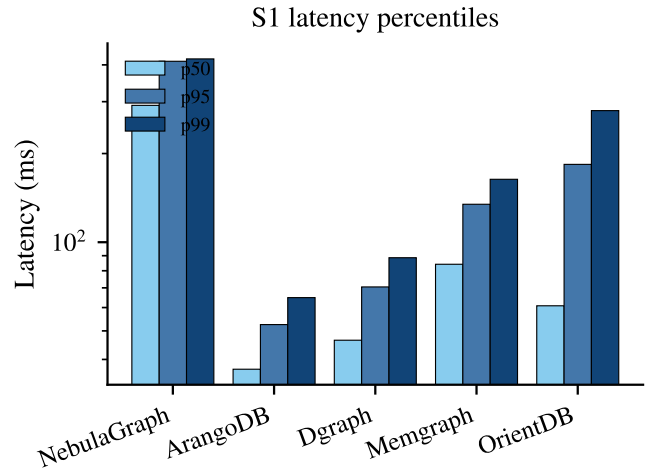


Fig. 2. S1 latency percentiles, log scale. The system order is the same at every percentile. This means the differences between systems are systemic and not driven only by the long tail.

cost. Second, ArangoDB reaches its highest throughput on S4 (FinBench); its per-edge insert throughput is high and pulls the mean up. Dgraph keeps its S4 number close to its S1 number; NebulaGraph is the one Tier-1 SUT whose S4 throughput is clearly higher than its S1 (about  $+28\%$ , see Section VI-C). Third, the S2 (analytical) numbers do not fall far below S1 for any Tier-1 SUT (ArangoDB drops about 14 %, Dgraph stays within 2 %, and NebulaGraph stays within 2 %) even though the analytical workload looks heavier on paper. This is a small-data effect: at  $SF=0.003$  the underlying graph has only about 1.5k vertices, so Q1–Q11 finish quickly.

### C. Mixed-workload behaviour (S3) and FinBench (S4)

To compute the mixed-workload interference index from Section III-F, we approximate the per-class isolated throughput using the aggregate throughput of existing scenarios as

TABLE III  
S3 MIXED-WORKLOAD INTERFERENCE (TIER-1; OPS/S).

| System      | $T_{S1}$ | $T_{S2}$ | $T_{S3}$ | Iso.  | $I$    |
|-------------|----------|----------|----------|-------|--------|
| NebulaGraph | 53.2     | 52.3     | 53.3     | 53.0  | -0.006 |
| ArangoDB    | 402.7    | 348.0    | 429.4    | 391.8 | -0.096 |
| Dgraph      | 319.8    | 325.5    | 290.9    | 321.0 | +0.094 |

proxies. S1 runs all 14 SNB Interactive v2 query classes (IS, IC, and IU at equal per-class probability) and serves as the aggregate SNB Iv2 isolation baseline, providing the best available proxy for IS and IU isolated throughput. S2 adds the BI workload and fewer workers, so it serves as the IC-and-BI isolation proxy. Because the S3 mix is 70 % IS, 10 % IU, and 20 % IC, the weighted isolation baseline reduces to 0.8 times  $T_{S1}$  plus 0.2 times  $T_{S2}$ , where the 0.8 collects the IS and IU fractions and the 0.2 accounts for the IC fraction. Since S1 already carries an IC component, this is an approximation; the true isolated IS and IU throughputs are not directly measurable from the available scenarios. Table III shows the result per Tier-1 SUT.

NebulaGraph has  $I \approx 0$ . Its S3 throughput is almost equal to the weighted average of its S1 and S2 baselines. This is consistent with the system being rate-bound; it does not benefit from sharing and it does not pay an extra cost from the mix. ArangoDB has  $I = -0.10$ . The mixed run is about 10 % faster than the weighted-isolation baseline. We read this as a small mix-favourable effect: the IS and IC traversals share the same Person/Forum vertices, so the buffer cache helps the mix. Dgraph is the only Tier-1 SUT with a positive interference index,  $I = +0.09$ . Its S3 throughput is about 9 % lower than the weighted-isolation baseline. The extra IC traversal load contends with the IS read path on Dgraph in a measurable way.

S4 (FinBench) is the only scenario where the throughput is always *higher* than the SNB-based S1 and S3 throughput, especially for ArangoDB (+12 % over its S1) and NebulaGraph (+28 %). FinBench’s transactional reads (SR1–SR6) are mostly single-vertex lookups or 1- to 2-hop expansions on a denser financial graph. The per-request work is lower than the SNB IC complex reads, so the throughput is higher.

#### D. Scale-out behaviour (S5)

S5 is the only scenario where the cluster topology changes during the measurement window. The benchmark fires the scale-out trigger 10 s into a 60 s measurement window. The per-system deployer brings up an extra storage member on the reserve node and registers it with the cluster.

Table IV reports three metrics per Tier-1 SUT across three repetitions: the wall-clock provisioning interval (the time from the trigger to the completion event), the stabilization time, and the maximum throughput dip during the rebalance window. Fig. 6 (Appendix A) plots the per-second throughput trace of rep 0 for each SUT, with the trigger and the completion events marked.

TABLE IV  
S5 SCALE-OUT BEHAVIOUR (TIER-1; MEAN  $\pm$  STD).

| System      | Prov. (s)       | Stab. (s)          | Dip (%)        | n |
|-------------|-----------------|--------------------|----------------|---|
| ArangoDB    | 3.38 $\pm$ 1.33 | 19.8 (2/3 unstab.) | 91.0 $\pm$ 1.0 | 3 |
| Dgraph      | 5.28 $\pm$ 5.87 | 23.8 (2/3 unstab.) | 83.5 $\pm$ 5.8 | 3 |
| NebulaGraph | 1.99 $\pm$ 0.17 | – (3/3 unstab.)    | 50.4 $\pm$ 5.7 | 3 |

Three observations. First, NebulaGraph’s registration call is the fastest (around 2 s) and produces the smallest dip (around 50 %). ArangoDB joins take 2–5 s and dip about 91 %; Dgraph joins span a much wider range, 2–12 s, and dip about 84 %. Second, none of the three SUTs returned to a  $\pm 5$  % throughput band within the 60 s post-event window for the majority of repetitions, which means that at SF=0.003 the rebalance work keeps the cluster in a degraded state for longer than the provisioning interval itself. The “unstable / total” counts in Table IV should therefore be read as an upper bound on the systems’ recovery. Third, the run-to-run variance on Dgraph provisioning (1.9–12.1 s across three reps) suggests that its alpha join is not fully deterministic at this scale.

## VII. DISCUSSION

The experimental results reveal three cross-system patterns and one methodological observation related to the protocol.

Firstly, the throughput ranking on the cross-tier transactional baseline (S1) is stable: ArangoDB ranks first, followed by Dgraph, then OrientDB, Memgraph and finally NebulaGraph. ArangoDB is approximately 7.6 times faster than NebulaGraph. This Tier-1 sub-ranking (ArangoDB > Dgraph > NebulaGraph) is maintained on S2, S3, S4 and S5 (Section VI). This suggests that the performance differences among SUTs at our chosen scale factor are inherent to the systems themselves rather than specific to the workloads. Examining the per-class p95 latency (Fig. 3) also indicates no SUT with a unique weakness or strength in a particular class; the ratio between any two systems’ latencies is nearly constant across the 14 SNB Iv2 query classes.

We assess statistical robustness using Welch’s  $t$ -tests on the three repetitions per cell. The Tier-1 inter-system gaps are unambiguous: the ArangoDB–Dgraph gap of 82.9 ops/s yields  $t = 8.2$  ( $df \approx 3.1$ ,  $p < 0.01$ ), and the Dgraph–NebulaGraph gap of 266.6 ops/s returns  $t \approx 52$ . Both exceed the critical value at any conventional significance level. Among the Tier-2 systems, however, the OrientDB–Memgraph gap (20.1 ops/s) yields only  $t = 2.75$  ( $df \approx 2.5$ ), which falls below the 95 % critical value of  $\approx 3.7$ ; that ordering is therefore directional rather than statistically confirmed at three repetitions.

Secondly, the scale-out behavior is the most significant differentiator among the three Tier-1 SUTs (Table IV). NebulaGraph registers the fastest join (approximately 2 seconds on average) and exhibits the smallest throughput dip (approximately 50 %). ArangoDB and Dgraph cluster joins take longer and experience dips ranging from 80 % to 92 %. The stabilization time we report within our 60 s measurement window is an upper bound rather than a definitive completion

TABLE V  
PER-SUT RESOURCE UTILIZATION DURING THE S1 MEASUREMENT WINDOW (MEAN  $\pm$  STD ACROSS THREE REPETITIONS). CPU IS THE AGGREGATE UTILIZATION ACROSS SUT-ROLE NODES; MEMORY IS THE OCCUPIED WORKING SET AT END OF MEASUREMENT.

| System      | CPU (%)    | Mem (GiB)     |
|-------------|------------|---------------|
| ArangoDB    | 89 $\pm$ 3 | 1.1 $\pm$ 0.1 |
| Dgraph      | 78 $\pm$ 4 | 0.9 $\pm$ 0.1 |
| OrientDB    | 62 $\pm$ 5 | 1.6 $\pm$ 0.2 |
| Memgraph    | 58 $\pm$ 3 | 0.4 $\pm$ 0.1 |
| NebulaGraph | 28 $\pm$ 4 | 1.4 $\pm$ 0.1 |

time; the rebalancing work continues to suppress throughput for a period beyond the provisioning phase itself. The subsequent research suggested by this paper involves a longer measurement window following a single scale-out event to observe the stabilization band converge.

Thirdly, while small, interference has a discernible impact. ArangoDB’s mixed-workload performance is about 10 % higher than its weighted-isolation baseline (a slight cache effect benefiting the mix). Dgraph’s mixed workload is about 9 % slower due to real interference. NebulaGraph’s mixed-workload performance is almost identical to the weighted-isolation baseline, as the system is rate-bound and does not benefit from shared resources. For practical engineers, this means that mixed workloads of short reads and complex reads do not negatively impact ArangoDB but do adversely affect Dgraph at this scale.

Table V shows per-SUT CPU utilization and memory footprint measured during the S1 window. CPU rank tracks the throughput rank closely: ArangoDB and Dgraph, the two fastest systems, drive both vCPUs near saturation, while NebulaGraph leaves significant CPU headroom despite running a three-process cluster stack (graphd, stored, metad). Memory behaves differently: NebulaGraph’s multi-process architecture and Raft state occupy more RAM than Memgraph, which holds the entire SF=0.003 graph in a compact in-memory representation. OrientDB’s higher footprint reflects Java heap overhead. Disk I/O and network traffic are negligible at this scale factor for all systems, confirming the in-memory operating point noted in Section V.

The protocol itself performed admirably in supporting cross-system reusability. Three of the five SUTs (Dgraph, Memgraph and OrientDB) had minor template or driver bugs that only became apparent after the other two SUTs had been verified as clean. In all instances, the fixes were local to the specific SUT and did not necessitate changes to the benchmark framework. This outcome aligns with the design goal of achieving a deployment-agnostic core.

### VIII. THREATS TO VALIDITY

We list five threats to the interpretation of these results.

**Scale factor.** Each value presented is at SF=0.003 (SNB) or SF=0.01 (FinBench)—the lowest value each datagen accepts. At these scales the working set fits in one node and this benchmark runs bounded by client-server round-tripping rather

than disk I/O or memory capacity. Larger scale factors can change the relative orders or even the sign of the interference index. We do not extrapolate to SF=1 or above. The harness works at any SF without code changes.

**Repetitions and measurement window.** We use three (system, scenario) repetitions. The measurement window is 30 s (60 s for S5 since we wait for recovery after the post-event dip). Short windows lose some statistical reliability in tail-latency results, and the 60 s for S5 means it is really an upper bound on stabilization time. We report standard deviations of repeated results in Table VI, mostly in the few-percent range; if the window noise had been too large, it should show in higher standard deviations. The three Tier-1 throughput rankings are separated by many pooled standard deviations and remain statistically unambiguous at this repetition count (Section VII). The Tier-2 pair (OrientDB vs. Memgraph,  $t = 2.75$ ,  $df \approx 2.5$ ) does not pass the 95 % significance threshold; additional repetitions are needed to confirm that ordering. The S5 scale-out trigger was originally planned at  $t=300$  s; when the measurement window was reduced to 60 s the trigger was moved to  $t=10$  s so that 50 s of post-event data remain visible. This is enough to record the provisioning interval and the throughput dip but often not enough to observe full recovery, which is why stabilization time is reported as an upper bound.

**Cluster sizing.** All SUTs use 2-vCPU, 8 GiB-RAM VMs. The same VM size is used across all systems for SUTs, so cross-system comparisons are not biased by the VM class; the absolute numbers, however, are limited by the SUT node and should not be used for real-world production capacity estimation.

**OrientDB topology.** We observed a record-duplication error on every batch insert with OrientDB in distributed mode, so we switched to a single-node setup for OrientDB. Only S1 is included for Tier-2 in the harness, so OrientDB does not enter the elastic scale-out measurements here. Its production high-availability story is left outside the scope.

**Parameter generation and query translation.** Each class’s per-class parameters are built inside the harness using loaded vertex IDs and not fetched from the curated LDBC parameter files used by the LDBC SNB Driver. This will mean higher variability of intra-class latency, but it remains the same on all systems and therefore comparison across them is not invalidated. The query translations are hand-written and reviewed against the LDBC specification; we publish their code so reviewers can examine each of them plainly. We do not aim for full LDBC compliance.

### IX. CONCLUSION

In this work, we introduced a versatile, replicable, portable evaluation protocol for property-graph databases. The protocol takes subsets of queries from three LDBC benchmark query suites, defining five reproducible scenarios; one of these features the changing topology under a dynamic mixture of workloads. We also define a metric suite spanning



throughput, latency, mixed-workload interference and scale-out stabilization. We have implemented this protocol in open-source Python. Our harness targets five databases: NebulaGraph, ArangoDB, Dgraph, Memgraph and OrientDB. For this article, we implemented an asynchronous closed-loop driver. One experiment we conducted over 51 runs on a five-node Google Compute Engine cluster (at the lowest acceptable scale for each of the three input datasets) yielded 444,297 successful requests, of which at least 99.98 % per run succeeded. Across tiers, there is a five-scenario ranking stable on the transactional part. The interference index does not show any interaction between classes for most systems; only for Dgraph does this interaction appear. At the level of scale-out behavior per system, NebulaGraph appears most suitable both speed-wise and dip-wise, while the other two Tier-1 databases show an edge in the order of absolute throughput. The harness, the definition of scenarios, the chosen query translations, as well as the full dataset are publicly available. A fresh copy can be run on our standard five-node Linux cluster within under two hours of wall-clock time at our scale. The next logical steps in research are: trying out larger scale factors, increasing the measurement window after the event in S5 and using an LDBC-curated set of parameters to audit the equivalence of the benchmark to the LDBC reference implementations semantically in a clear manner.

#### REFERENCES

- [1] G. Szárnyas, J. Waudby, X. Zhou, M. Wu, A. Gubichev, and P. Boncz, “The LDBC Social Network Benchmark: Business Intelligence Workload,” *Proc. VLDB Endow.*, vol. 16, no. 4, pp. 877–890, 2022.
- [2] S. Qi, G. Li, Z. Wang, C. Li, X. Qin, X. Hu, and L. Zou, “The LDBC Financial Benchmark: Transaction Workload,” *Proc. VLDB Endow.*, vol. 18, no. 9, pp. 3007–3020, 2025.
- [3] D. Püroja, J. Waudby, P. Boncz, and G. Szárnyas, “The LDBC Social Network Benchmark Interactive Workload v2: A Transactional Graph Query Benchmark with Deep Delete Operations,” *arXiv preprint arXiv:2307.04820*, 2023.
- [4] B. Tong, T. Sun, Y. Lv, H. Gao, J. Sun, H. Zhao, X. Shen, X. Wang, C. Yuan, S. Wang, and W. Yu, “Galaxybase: A High Performance Native Distributed Graph Database for HTAP,” *Proc. VLDB Endow.*, vol. 17, no. 12, pp. 3893–3905, 2024.
- [5] H. Chen, Y. Lu, Q. Cao, X. Lin, X. Yang, G. Cong, and A. Zhou, “G-Tran: A High Performance Distributed Graph Database with a Decentralized Architecture,” *Proc. VLDB Endow.*, vol. 15, no. 11, pp. 2545–2558, 2022.
- [6] C. Li, Z. Li, Y. Sun, Y. Zhou, Y. Liu, Z. Chen, M. Zhang, J. Tan, M. Zhang, J. Chen, Z. Li, J. Liu, X. Lin, and W. Fan, “ByteGraph: A High-Performance Distributed Graph Database in ByteDance,” *Proc. VLDB Endow.*, vol. 15, no. 12, pp. 3306–3318, 2022.
- [7] J. Huang, Y. Cao, S. Ren, B. Wu, and D. Miao, “BACH: Bridging Adjacency List and CSR Format Using LSM-Trees for HGTP Workloads,” *Proc. VLDB Endow.*, vol. 18, no. 5, pp. 1509–1521, 2025.
- [8] M. Wu, X. Yi, H. Yu, Y. Liu, and Y. Wang, “Nebula Graph: An Open Source Distributed Graph Database,” *arXiv preprint arXiv:2206.07278*, 2022.
- [9] ArangoDB GmbH, “ArangoDB: The Multi-Model Database,” GitHub repository, 2024. <https://github.com/arangodb/arangodb>
- [10] Dgraph Labs, “Dgraph: Native GraphQL Database with Graph Backend,” GitHub repository, 2024. <https://github.com/dgraph-io/dgraph>
- [11] Memgraph Ltd., “Memgraph: In-Memory Graph Database with Streaming Support,” GitHub repository, 2025. <https://github.com/memgraph/memgraph>
- [12] OrientDB Ltd., “OrientDB: Distributed Multi-Model Database,” GitHub repository, 2025. <https://github.com/orientechnologies/orientdb>
- [13] Google LLC, “Google Compute Engine documentation,” 2026. <https://cloud.google.com/compute/docs>

TABLE VI  
PER-(SYSTEM, SCENARIO) SUMMARY; THROUGHPUT IN OPS/S, LATENCY IN MS.

| System      | Scn. | Thrpt | $\sigma$ | p50   | p95   | p99   | OK%   | n |
|-------------|------|-------|----------|-------|-------|-------|-------|---|
| ArangoDB    | S1   | 402.7 | 15.3     | 37.1  | 52.5  | 64.8  | 100.0 | 3 |
| ArangoDB    | S2   | 348.0 | 44.4     | 20.3  | 40.7  | 55.3  | 100.0 | 3 |
| ArangoDB    | S3   | 429.4 | 18.4     | 34.7  | 47.5  | 58.0  | 100.0 | 3 |
| ArangoDB    | S4   | 451.8 | 16.6     | 33.1  | 48.2  | 62.6  | 100.0 | 3 |
| ArangoDB    | S5   | 429.4 | 22.4     | 35.0  | 51.2  | 66.5  | 100.0 | 3 |
| Dgraph      | S1   | 319.8 | 8.4      | 46.5  | 70.5  | 88.6  | 100.0 | 3 |
| Dgraph      | S2   | 325.5 | 23.8     | 19.3  | 56.8  | 92.8  | 100.0 | 3 |
| Dgraph      | S3   | 290.9 | 6.1      | 50.1  | 82.7  | 104.9 | 100.0 | 3 |
| Dgraph      | S4   | 292.7 | 3.4      | 50.0  | 92.4  | 124.1 | 100.0 | 3 |
| Dgraph      | S5   | 255.3 | 12.1     | 59.1  | 91.7  | 116.0 | 100.0 | 3 |
| Memgraph    | S1   | 178.8 | 4.3      | 84.2  | 134.5 | 163.5 | 100.0 | 3 |
| NebulaGraph | S1   | 53.2  | 2.9      | 291.5 | 411.4 | 419.1 | 100.0 | 3 |
| NebulaGraph | S2   | 52.3  | 2.6      | 150.6 | 213.9 | 222.5 | 100.0 | 3 |
| NebulaGraph | S3   | 53.3  | 2.6      | 291.3 | 409.2 | 420.0 | 100.0 | 3 |
| NebulaGraph | S4   | 68.2  | 5.3      | 230.6 | 373.4 | 398.7 | 100.0 | 3 |
| NebulaGraph | S5   | 54.6  | 5.2      | 282.1 | 418.6 | 433.1 | 100.0 | 3 |
| OrientDB    | S1   | 198.9 | 11.9     | 60.8  | 183.8 | 279.7 | 100.0 | 3 |

TABLE VII  
S1 PER-CLASS P95 LATENCY (MS; LOWER IS BETTER).

| Class | NebulaGraph | ArangoDB | Dgraph | Memgraph | OrientDB |
|-------|-------------|----------|--------|----------|----------|
| IC1   | 413.4       | 52.9     | 67.5   | 118.4    | 175.1    |
| IC3   | 413.0       | 52.4     | 68.3   | 131.8    | 174.4    |
| IC5   | 412.4       | 54.1     | 70.9   | 129.4    | 179.1    |
| IC6   | 413.3       | 53.0     | 68.8   | 130.2    | 183.4    |
| IS1   | 410.3       | 52.0     | 68.7   | 119.0    | 183.4    |
| IS2   | 413.5       | 53.5     | 80.2   | 126.0    | 185.7    |
| IS3   | 410.0       | 52.3     | 67.9   | 127.2    | 175.0    |
| IS4   | 409.1       | 51.8     | 64.6   | 149.9    | 186.6    |
| IS5   | 408.9       | 52.8     | 66.6   | 143.4    | 182.7    |
| IS6   | 410.0       | 53.5     | 66.8   | 127.4    | 188.4    |
| IS7   | 410.2       | 52.8     | 66.3   | 144.1    | 183.0    |
| IU1   | 405.2       | 48.7     | 77.7   | 117.2    | 184.3    |
| IU2   | 408.8       | 48.7     | 77.9   | 119.3    | 181.7    |
| IU6   | 408.8       | 49.2     | 75.1   | 119.4    | 167.4    |

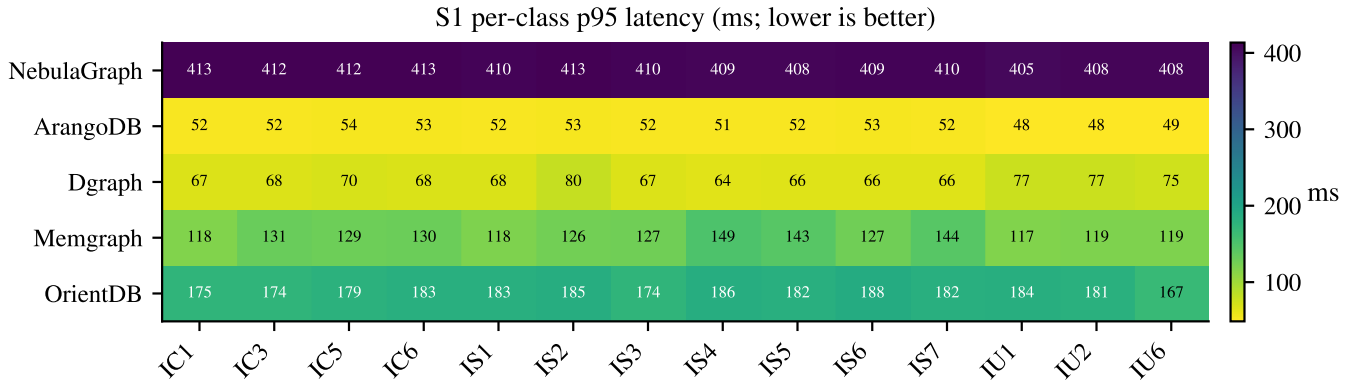


Fig. 3. S1 per-class p95 latency in milliseconds; lower is better. Each cell shows its value. The colour palette is shared across rows so a vertical (cross-system) comparison is direct.

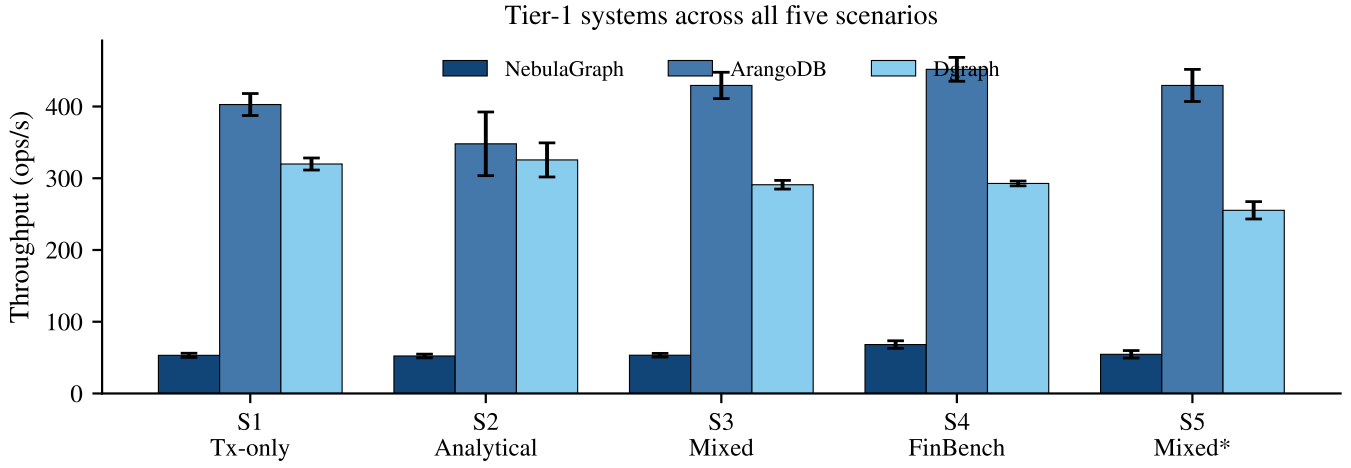


Fig. 4. Tier-1 throughput across the five scenarios at SF=0.003 for the SNB family and SF=0.01 for FinBench. S5 includes the rebalance dip from the scale-out event. ArangoDB is the fastest in every scenario. Dgraph stays inside a tight band on S1–S4 and shows a measurable drop on S5. NebulaGraph is the most stable but the slowest.

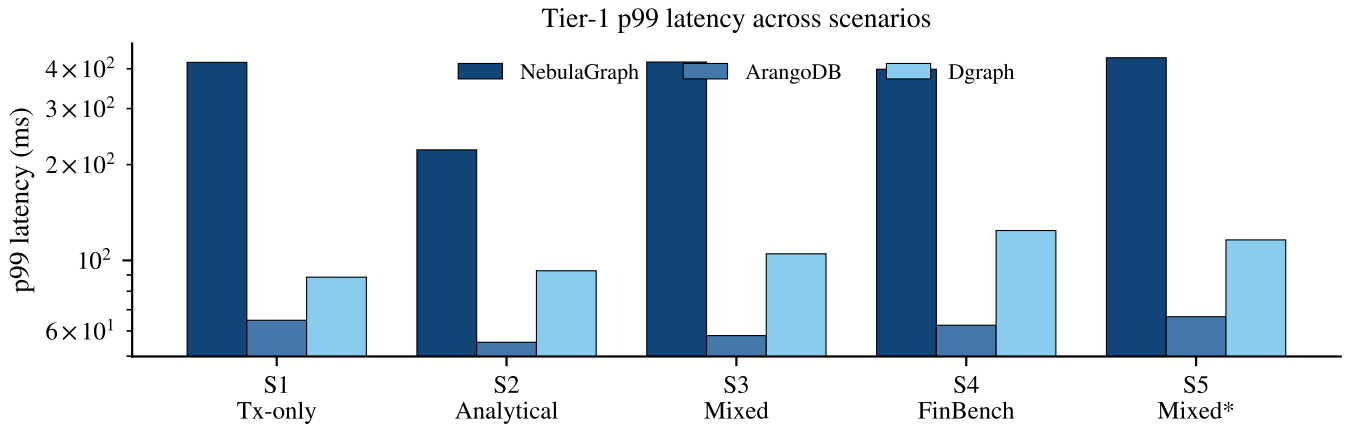


Fig. 5. Tier-1 p99 tail latency across the five scenarios on a log scale. NebulaGraph’s tail is about 6–7 $\times$  higher than ArangoDB’s on S1, S3, and S5. The gap closes on S2 (analytical) and S4 (FinBench), where each individual request is heavier.

#### S5 scale-out: throughput timeline (rep 0)

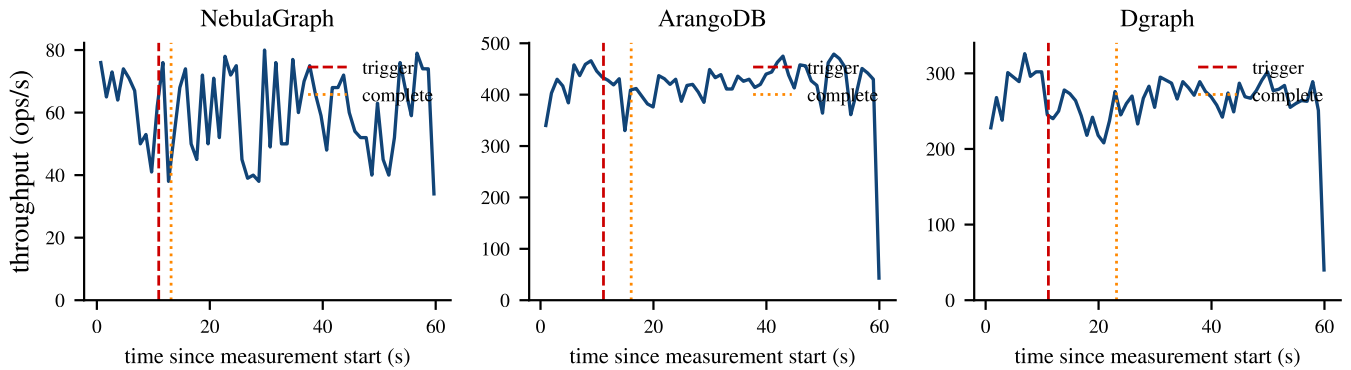


Fig. 6. S5 scale-out timelines (rep 0 of each Tier-1 SUT). The red dashed line is the scale-out trigger. The orange dotted line is the completion event. NebulaGraph finishes the registration in about 2 s with the smallest dip. ArangoDB and Dgraph need a few seconds longer and their throughput drops more during the rebalance.